

Code Explanation

Test Configuration and Fixtures for ETL Pipeline Testing

This code defines test configurations and fixtures for ETL pipeline testing using PySpark. It provides a SparkSession and sample data for sales, products, and customers.

Overview

The code is written in Python and utilizes the Pytest framework for testing. It defines several fixtures that provide a SparkSession and sample data for testing ETL pipelines. The fixtures are designed to be reusable across multiple test cases.

Step 1: Defining the SparkSession Fixture

The first step is to define a fixture that creates a SparkSession for testing. This fixture is used to provide a SparkSession to other fixtures and test cases.

```
@pytest.fixture(scope="session")
def spark():
    """Create a SparkSession for testing."""
    return (SparkSession.builder
            .master("local[2]")
            .appName("ETL Pipeline Tests")
            .config("spark.sql.warehouse.dir", "file:///tmp/spark-warehouse")
            .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
            .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())
```

Step 2: Defining Schemas for Sales, Products, and Customers Data

The next step is to define fixtures that provide schemas for sales, products, and customers data. These schemas are used to create DataFrames with the correct structure.

```
@pytest.fixture
def sales_schema():
    """Schema for sales data."""
    return StructType([
        StructField("sale_id", StringType(), False),
        StructField("date", StringType(), False),
        StructField("customer_id", StringType(), True),
        StructField("product_id", StringType(), False),
        StructField("quantity", IntegerType(), True),
        StructField("unit_price", DoubleType(), True)
    ])

@pytest.fixture
def products_schema():
    """Schema for products data."""
    return StructType([
        StructField("product_id", StringType(), False),
        StructField("product name", StringType(), False),
```

```

        StructField("category", StringType(), True),
        StructField("unit_cost", DoubleType(), True)
    ])

@pytest.fixture
def customers_schema():
    """Schema for customers data."""
    return StructType([
        StructField("customer_id", StringType(), False),
        StructField("customer_name", StringType(), False),
        StructField("region", StringType(), True),
        StructField("segment", StringType(), True)
    ])

```

Step 3: Creating Sample Data for Sales, Products, and Customers

The final step is to define fixtures that create sample data for sales, products, and customers using the schemas defined earlier.

```

@pytest.fixture
def sample_sales_data(spark, sales_schema):
    """Sample sales data for testing."""
    data = [
        ("S001", "2023-10-01", "C001", "P001", 2, 10.0),
        ("S002", "2023-10-01", "C002", "P002", 1, 25.0),
        ("S003", "2023-10-02", "C001", "P003", 3, 15.0),
        ("S004", "2023-10-02", "C003", "P001", 1, 10.0),
        ("S005", "2023-10-03", "C002", "P002", 2, 25.0),
        ("S006", "2023-10-03", None, "P003", None, 15.0)
    ]
    return spark.createDataFrame(data, schema=sales_schema)

@pytest.fixture
def sample_products_data(spark, products_schema):
    """Sample products data for testing."""
    data = [
        ("P001", "Product 1", "Category A", 5.0),
        ("P002", "Product 2", "Category B", 12.5),
        ("P003", "Product 3", "Category A", 7.5)
    ]
    return spark.createDataFrame(data, schema=products_schema)

@pytest.fixture
def sample_customers_data(spark, customers_schema):
    """Sample customers data for testing."""
    data = [
        ("C001", "Customer 1", "East", "Retail"),
        ("C002", "Customer 2", "West", "Wholesale"),
        ("C003", "Customer 3", "East", "Retail")
    ]
    return spark.createDataFrame(data, schema=customers_schema)

```